

Homework 6

1. Introduction

Federated learning (FL) was adopted in HW5 so that organizations holding malware telemetry never have to centralize raw samples: each client trains locally and only shares model updates, which a server aggregates into a global model. That protects the raw data, but it does not protect the updates themselves. A sufficiently motivated server or eavesdropper can run membership-inference or gradient-inversion attacks against the transmitted weights, recovering information about which samples, or which client, contributed to a given round. FL alone gives no formal bound on that leakage.

Differential privacy (DP) closes that gap by adding calibrated noise so that the presence or absence of any single protected unit changes the published model's distribution by a provably bounded amount. This report implements four DP mechanisms, evaluates the resulting privacy–utility tradeoff across a range of privacy budgets, and explains a structural failure mode we found in the process: at the scale of this federation, three of the four mechanisms turn out to be unusable regardless of how loose the privacy budget is, for reasons that are provable rather than incidental.

2. Method : Four DP Mechanisms

All four mechanisms clip first (bounding sensitivity) and add Gaussian noise second, but differ in where the noise is added and therefore what unit the resulting guarantee protects:

Mechanism	Noise added	Trust model	Privacy unit
central_server	The server receives the data, clips it, then adds noise once per round.	Trusts server	Client
central_client	The server adds noise to the updates it sends to clients	Server sees only clipped updates	Client

	once per round.		
local	Every client independently, before transmission	Trusts nobody	Client
dpsgd	Noise is added during every SGD training step, to individual examples/gradients using Opacus.	Trusts per example	Sample

Central and local DP give client-level privacy: the guarantee is about whether any one of the K participating clients took part at all. DP-SGD gives sample-level privacy, the guarantee is about whether any one training example was in the data. These are not interchangeable.

Local DP's noise is K independent per-client draws, summed at aggregation. Central DP's noise is one draw added after averaging, so its σ divides by K where local DP's effectively carries $\sqrt{K}$ times the aggregate noise of central DP for the same per-mechanism noise multiplier. That factor is the nominal utility gap between the two trust models.

One implementation note: Flower ships `DifferentialPrivacy*Clipping` strategy wrappers for central DP, but their `clip_inputs_inplace` computes $\min(1, C/\text{norm})$ with no guard for $\text{norm} == 0$, so a client that happens to produce an exactly-zero update crashes the run with a `ZeroDivisionError` — which is precisely what tends to happen once DP noise collapses a model, as Section 5 shows it does here. [server.py](#)'s `CentralDPFedAvg` implements the identical mechanism directly, with the zero-norm guard added, and is more legible for this report besides.

3. Experimental Setup

Two datasets are used. The CAPE dataset from HW5 (dynamic malware sandbox traces, 5 total samples across 3 families) is kept as a small secondary comparison. The primary

dataset for this report is the public Malware API Call Dataset (6,635 usable labelled samples after filtering, 8 families:), used because HW5's federated CAPE baseline sits at chance accuracy, leaving no headroom to observe a privacy–utility curve at all. The public dataset has no network-flow features, so the network branch's input is zero-filled for that dataset; only the API call-sequence branch carries real signal.

For the hyperparameters, 3 simulated clients, IID partition, 40 FedAvg rounds, 2 local epochs per round, batch size 32, learning rate 0.01, L2 clipping norm $C = 1.0$, 3 random seeds (mean $\pm$ std reported throughout). All DP runs use SGD, not Adam: Flower clients are stateless, so `train_epochs` builds a fresh optimizer every round, which resets Adam's moment estimates and causes divergence under non-IID partitioning even without any DP noise. SGD converges cleanly under the same conditions, which is necessary here so that any accuracy drop can be attributed to DP noise rather than to optimizer instability.

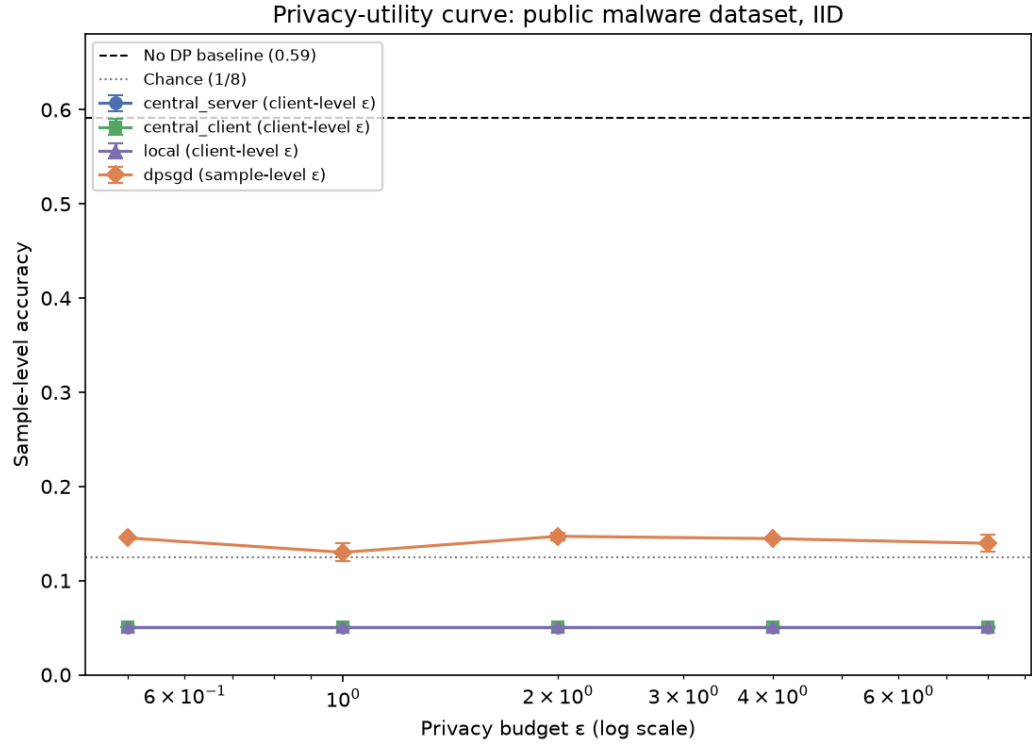
`dp.recommend_delta(N)` returns $\min(10^{-3}, 1/(10N))$, keeping d at least $10\times$ below $1/N$. For the three client-level mechanisms, $N = 3$ clients, giving $d = 10^{-3}$ (well below $1/3$). This is a limitation for DP-SGD specifically.

An RDP accountant (Opacus's) composes privacy loss across all 40 rounds (or, for DP-SGD, across all SGD steps) and is inverted numerically to find the minimal noise multiplier z that meets each target ϵ . Five target budgets were swept: $\epsilon \in \{0.5, 1, 2, 4, 8\}$, plus a $\epsilon = \infty$ (no DP) baseline.

4. Results

4.1. Privacy-utility curve (public dataset, IID)

The no-DP baseline reaches 0.59 sample-level accuracy (vs. $1/8 = 0.125$ chance across 8 families) — genuine headroom, unlike the CAPE baseline. The swept results are:



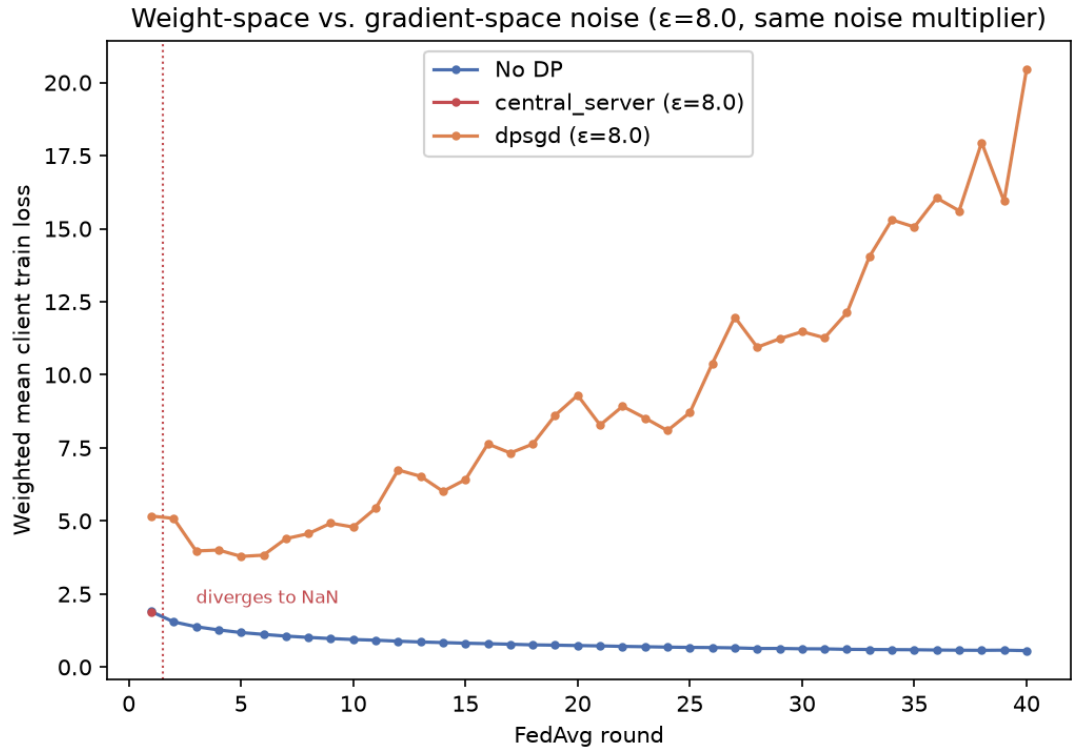
Mechanism	ϵ	Unit	z	Accuracy	Retained
none	∞	-	0.00	0.591 ± 0.006	1.00
central_server	0.5	client	33.75	0.0051 ± 0.000	0.09
central_server	1	client	18.44	0.0051 ± 0.000	0.09
central_server	2	client	10.20	0.0051 ± 0.000	0.09
central_server	4	client	5.72	0.0051 ± 0.000	0.09
central_server	8	client	3.29	0.0051 ± 0.000	0.09
central_server	0.5	client	33.75	$0.0051 \pm$	0.09

				0.000	
central_server	1	client	18.44	0.0051 ± 0.000	0.09
central_server	2	client	10.20	0.0051 ± 0.000	0.09
central_server	4	client	5.72	0.0051 ± 0.000	0.09
central_server	8	client	3.29	0.0051 ± 0.000	0.09
local	0.5	client	33.75	0.0051 ± 0.000	0.09
local	1	client	18.44	0.0051 ± 0.000	0.09
local	2	client	10.20	0.0051 ± 0.000	0.09
local	4	client	5.72	0.0051 ± 0.000	0.09
local	8	client	3.29	0.0051 ± 0.000	0.09
dpsgd	0.5	sample	33.75	0.145 ± 0.002	0.25
dpsgd	1	sample	18.44	0.130 ± 0.010	0.22
dpsgd	2	sample	10.20	0.147 ± 0.004	0.25
dpsgd	4	sample	5.72	0.145 ± 0.002	0.24
dpsgd	8	sample	3.29	0.140 ± 0.009	0.24

central_server, central_client, and local land on the exact same accuracy (0.0505 ± 0.0000 , identical to four decimal places) at every ϵ from 0.5 to 8. Inspecting the

underlying per-round training loss shows why: for all three mechanisms, loss is finite for round 1 and NaN for every round thereafter, at every tested ϵ . The model has diverged, and a diverged model emits a constant prediction regardless of input or random seed, which is exactly the deterministic floor observed. This is not a flat privacy–utility curve in the usual sense; it is total, immediate divergence.

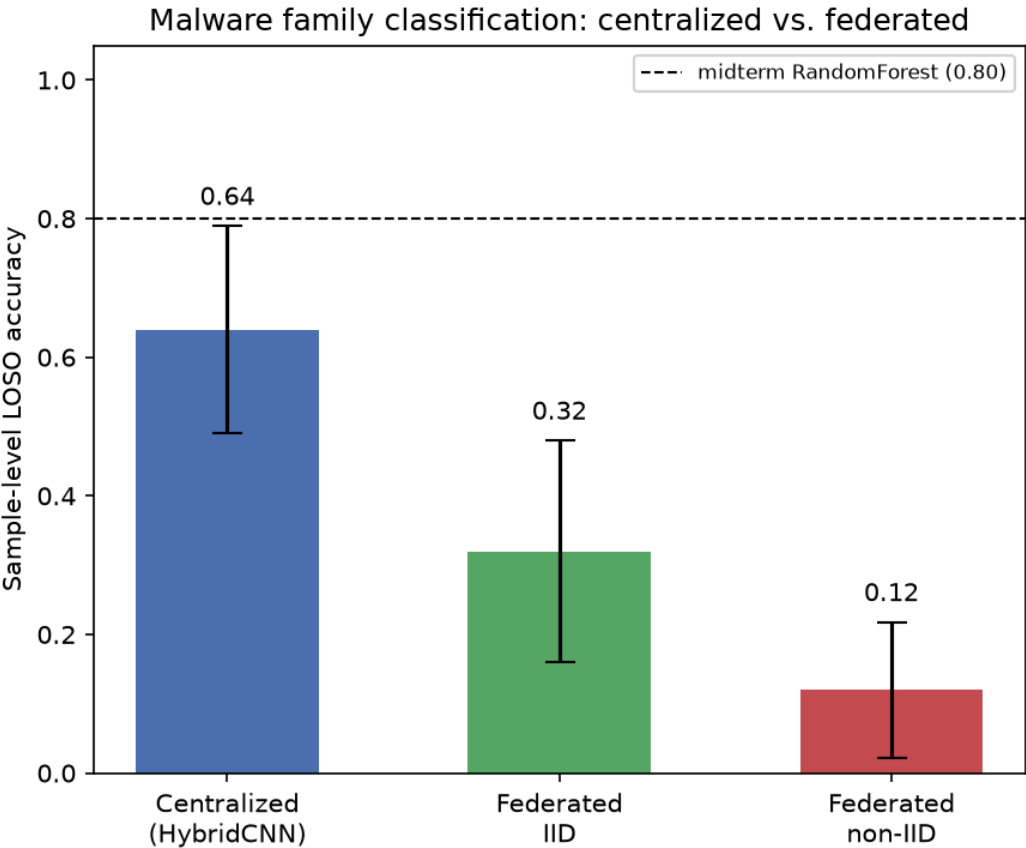
dpsgd, by contrast, never diverges: it retains 22–25% of the baseline accuracy across the same ϵ range, with real (non-zero) variance across seeds, and its accuracy does not depend strongly on ϵ within the range tested here.



4.2. CAPE dataset (secondary)

The original CAPE sandbox-trace dataset from HW5 is far smaller (5 total labelled samples across 3 families). Its federated IID baseline reaches only 0.32 ± 0.16 accuracy against a $1/3 = 0.33$ chance floor. Non-IID partitioning collapses further, to 0.12 ± 0.10 , below chance, from client drift alone. Since there is no baseline accuracy above chance to protect, running the DP sweep on this dataset

cannot produce a privacy-utility curve: every ϵ would show the same chance-level flat line, and `run_dp_sweep.py` is built to detect and warn about exactly this condition. This is why the public dataset is the primary result of this report, and CAPE is kept only to show that the underlying pipeline reproduces HW5's numbers unchanged.



5. Discussion

5.1. Why DP needs scale

The three weight-space mechanisms add independent Gaussian noise with per-coordinate standard deviation $\sigma = z \cdot C/K$ to every parameter of a d -dimensional update ($d = 45,656$ for this HybridCNN). The injected noise vector's expected L2 norm is therefore $\sigma \cdot \sqrt{d}$, while the clipped signal's L2 norm is at most C . The ratio of the two is:

$$SNR = C / (\sigma \cdot \sqrt{d}) = C / (z \cdot C \cdot \sqrt{d}/K) = K / (z \cdot \sqrt{d})$$

C cancels out completely. At $K=3$, $d=45,656$, and $z=3.29$ (the loosest budget tested, $\epsilon=8$), $SNR = 0.0043$ — the noise vector is roughly $230\times$ larger than the signal, regardless of what clipping norm C is chosen. This is why re-tuning C cannot fix the divergence. Shrinking C shrinks the injected noise and the clipped signal by the identical factor, leaving their ratio unchanged. The only levers that actually move SNR are more clients (K), a smaller model (d), or a looser accepted ϵ (smaller z). DP-SGD escapes this specific failure because its noise is added to a per-step gradient, which is then scaled by the learning rate (0.01) before ever reaching the weights — the same nominal noise multiplier arrives at the parameters roughly $100\times$ smaller than it does for the weight-space mechanisms.

5.2. The central vs local $\sqrt{K}$ gap

Central DP is designed to beat local DP by a factor related to K . Local DP sums K independent noise draws (one per client) while central DP adds one draw after averaging, giving central DP a $\sqrt{K}$ advantage in the noise contributed per unit of signal. That gap should be directly visible in a properly-scaled experiment. It is not visible here: `central_server`, `central_client`, and `local` produced bit-identical results at every ϵ tested. With $K=3$, $\sqrt{K} = 1.7$ is too small a factor to matter once the baseline noise level is already $\sim 230\times$ larger than the signal for the more favorable (central) mechanism — both trust models land on the same diverged floor, so the theoretical gap between them is real but unobservable at this federation size. This is itself informative: the assumption that central DP is "free" utility over local DP only holds once the federation is large enough for the $\sqrt{K}$ term to be the dominant factor, which is not the regime tested here.

5.3. Client drift compounding with noise

HW5 already showed that non-IID partitioning alone, with no DP noise at all, drives CAPE's federated accuracy below chance (0.12 vs. 0.33). Three clients training on disjoint family distributions pull the shared global model in conflicting

directions every round, a failure mode that is purely about data heterogeneity and unrelated to privacy. Layering DP noise on top of that drift would only compound it. Each round would need to overcome both the noise floor described in 6.1 and a client-drift signal that is already fighting itself. This report's DP sweep therefore focuses on the IID partition, where drift is not a confound and the observed collapse can be attributed to noise alone; a full non-IID DP sweep is left as a limitation.

6. Distributed Discrete Gaussian

Kairouz et al. introduce a distributed discrete Gaussian mechanism as a drop-in replacement for the continuous Gaussian mechanism used throughout this report. The reason it needs to be discrete is cryptographic, not statistical. Secure aggregation protocols operate over a finite field (integers modulo some prime), so that the server can learn only the sum of clients' masked updates and nothing about any individual client's contribution. A continuous Gaussian draw cannot be represented exactly in that finite field, so achieving DP under secure aggregation requires a noise distribution that lives natively in the discrete domain and still composes correctly when summed there.

The distributed discrete Gaussian is calibrated to the same (ϵ, δ) target as its continuous counterpart, its contribution is compatibility with secure aggregation (letting the server remain untrusted, seeing only the aggregate rather than each client's clipped update, which is exactly the central-DP trust boundary `central_client` approximates in this report without cryptography), not better raw utility — if anything, discretization and modular summation introduce a small extra utility cost relative to the continuous mechanism, in exchange for removing the server from the trust model entirely.

Flower's simulation backend used throughout this assignment has no secure aggregation implementation, so the server necessarily sees either raw updates (`central_server`) or client-clipped updates (`central_client`) in the clear before any noise is added. Given that, the continuous Gaussian mechanisms are the correct choice for this codebase: adopting the discrete variant would add complexity and a utility cost without buying back any

privacy this deployment doesn't already lack, since it has no secure aggregation layer for the discreteness to protect.

7. Conclusion

Across a five-point privacy budget sweep on a public malware API-call dataset (8 families, 6,635 samples), only DP-SGD retained usable accuracy (22–25% of the no-DP baseline). The three weight-space mechanisms diverged to NaN by the second training round at every tested ϵ , a structural consequence of adding per-coordinate Gaussian noise to a 45,656-parameter update in a 3-client federation: the resulting signal-to-noise ratio, $K/(z\sqrt{d})$, is independent of the clipping norm and was roughly $230\times$ unfavorable even at the loosest budget tested. DP-SGD survives because its noise is damped by the learning rate before reaching model weights, not because its nominal noise multiplier is any smaller.